

Model Paper - Database Management Systems

Generated: 2026-03-05 02:05:57

Q1 (20 marks)

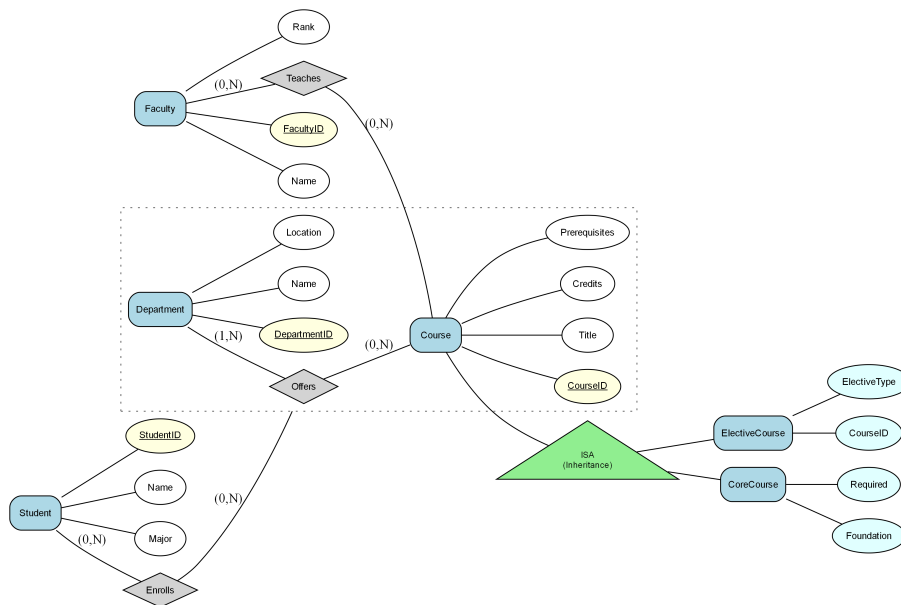
Consider the following Enhanced Entity-Relationship (EER) diagram:

Note: The diagram uses (min, max) cardinality notation where:

- (1,1) indicates one-to-one relationship (each entity participates exactly once)
- (1,N) or (1,*) indicates one-to-many relationship (one entity can relate to many)
- (0,1) indicates optional participation (zero or one)
- (0,N) or (0,*) indicates optional many participation (zero or many)

In a university database, the entity 'Department' has attributes: DepartmentID, Name, and Location. The entity 'Course' includes attributes: CourseID, Title, Credits, and Prerequisites. The relationship 'Offers' connects 'Department' and 'Course', where each department can offer multiple courses. The entity 'Student' possesses attributes: StudentID, Name, and Major, while the relationship 'Enrolls' links 'Student' to 'Course' with an attribute EnrollmentDate. Additionally, the entity 'Faculty' has attributes: FacultyID, Name, and Rank. The 'Teaches' relationship connects 'Faculty' to 'Course', allowing for multiple faculty members teaching the same course. The 'Course' entity is specialized into 'ElectiveCourse' and 'CoreCourse', with 'ElectiveCourse' having an attribute ElectiveType, and 'CoreCourse' including attributes: Required and Foundation. This EER diagram includes a situation where a relationship between ['Department', 'Course'] (for example, 'Offers') is treated as a single unit when relating to Student via 'Enrolls'. Model this using an aggregation so that the external relationship is between the external entity and the combined Department-Course-Offers structure, not directly to only one inner entity. A Department can be related to many Courses through the 'Offers' relationship (Course participation is optional). A Student can be related to many Courses, and a Course can be related to many Students through the 'Enrolls' relationship (Student participation is optional, Course participation is optional). A Faculty can be related to many Courses, and a Course can be related to many Faculty through the 'Teaches' relationship (Faculty participation is optional, Course participation is optional)

Figure: EER



a) Convert the following EER model into the relational model. Indicate the primary keys and the foreign keys of the resulting relations clearly. (17 marks)

b) What is the best option for mapping the ISA hierarchies in the diagram? Justify your answer. (3 marks)

Q2 (15 marks)

Consider a relation $R(A, B, C, D, E)$ with the following set of functional dependencies F over R :

$F = \{ \{ AB \rightarrow C, B \rightarrow D, CD \rightarrow E, E \rightarrow A, AC \rightarrow B \} \}$

a) Find all the keys in relation R using the attribute closure method. (4 marks)

b) Is R in 3NF? Give reasons for your conclusion. (3 marks)

c) Is R in BCNF? Give reasons for your conclusion. If R is not in BCNF, convert it to a set of BCNF relations. (8 marks)

Q3 (25 marks)

A university has designed a database to manage its student information system. The schema includes the following relation: $Courses(course_id: string, title: string, credits: integer, department: string)$. The university needs to ensure that certain conditions are met for the data it stores, thus implementing various integrity constraints to maintain accuracy and consistency throughout the database system.

a) In Type 2 Driver, JDBC API calls are converted to native Java API calls. Accept or refute the above statement justifying your answer. (2 marks)

b) Code Segment:

```
```java
String sql = "SELECT * FROM employees WHERE department = ?";
PreparedStatement pstmt = connection.prepareStatement(sql);
pstmt.setString(1, "IT");
ResultSet rs = pstmt.executeQuery();
```
```

Which type of JDBC statement is used in the code segment shown above?

Briefly explain when this type of statement will be used. (2 marks)

c) Briefly explain the different types of integrity constraints that can be implemented in a relational database schema. (2 marks)

d) Briefly explain how foreign key constraints can enforce referential integrity in the context of a student enrollment database. (3 marks)

e) A university is establishing role-based access control for its database system that manages Courses, Students, and Enrollments. Sarah is the senior database administrator responsible for overall database administration. Emily is a junior database administrator responsible for managing the Enrollments database. Nathan is a database developer responsible for creating and maintaining database objects. Michael is a data entry operator responsible for entering and updating student enrollment data.

i. Write a T-SQL statement to create a login to Sarah with windows authentication. (2 marks)

ii. Provide Sarah with the necessary permissions to handle all administrative tasks within the server using a fixed server role. (3 marks)

iii. Assuming Emily's username is 'emily.e', write T-SQL statements to assign her the responsibility of managing the Enrollments database using a user-defined server role. (5 marks)

iv. Assuming Nathan's login name is 'nathan.n', write T-SQL statements to grant him permission to create tables and databases. (3 marks)

v. Assuming Michael's login name is 'michael.m', write T-SQL statements to allow him to perform data entry tasks on the Students and Enrollments databases. (3 marks)

Q4 (40 marks)

Consider the following schema of a database designed for a Hospital: Patient (patientId: int, name: varchar(100), address: varchar(150), dob: date) Appointment (appointmentId: int, appointmentDate: date, status: varchar(50), patientId: int) Fee (feeId: int, amount: real, paymentStatus: varchar(50), appointmentId: int) Doctor (doctorId: int, name: varchar(100), specialty: varchar(50), patientId: int) The 'Patient' table stores information about patients, including unique patient ID, name, address, and date of birth. The 'Appointment' table stores details about appointments, including a unique appointment ID, appointment date, status, and associated patient ID. The 'Fee' table contains appointment fee information, including unique fee ID, fee amount, payment status, and associated appointment ID. The 'Doctor' table lists doctors along with their unique ID, name, specialty, and the patient they are treating. The 'Doctor' table contains details about doctors, including unique doctor ID, name, specialization, and contact information.

a) Write SQL Queries to perform the following:

i. Find the name of patients who are linked to appointment records where appointmentDate is 'SampleValue'. (4 marks)

ii. Find the patient who has the highest total amount compared to all other patients. Find the patientId and name. (6 marks)

iii. Find the appointmentDate, status, and name from Appointment, Patient, and Appointment where Appointment.appointmentDate is not null. (7 marks)

- b) b) Create a function to calculate the total fee amount for a given patient. This function should account for fee with a late penalty of 10% applied to those marked as 'overdue' payment status. (11 marks)
- c) c) Create a trigger that automatically updates the 'amount' column in the 'Fee' table whenever a new fee record is added, or an existing fee record is updated. The 'amount' column should contain the total fee amount for each patient. Ensure the trigger adjusts the 'amount' column accurately to reflect any partial payments made by patient. The patient pays only 50% of the total fee amount as an initial payment when the payment status is marked as 'Partial'. (12 marks)